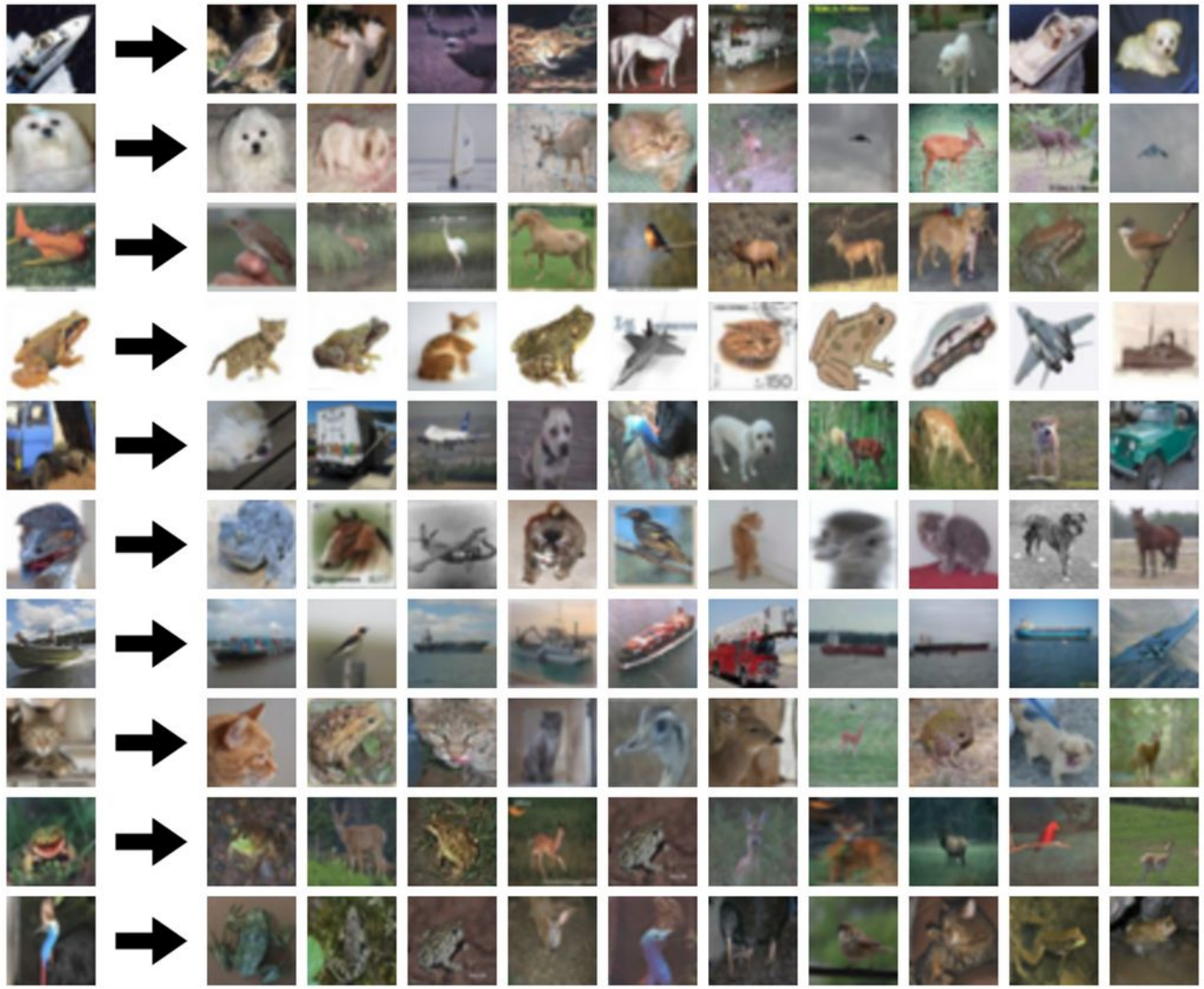
Lecture 2: Image Classification

Part 2: k-NN, hyperparameters, and validation

Week 1 recap

Image classification	A data driven classifier learns from labeled examples.
Nearest neighbor	Prediction uses the label of the closest training example.
Computational cost	Fast training comes with expensive prediction over the stored dataset.
Today	Decision boundaries motivate k-NN, followed by distance metrics and validation.

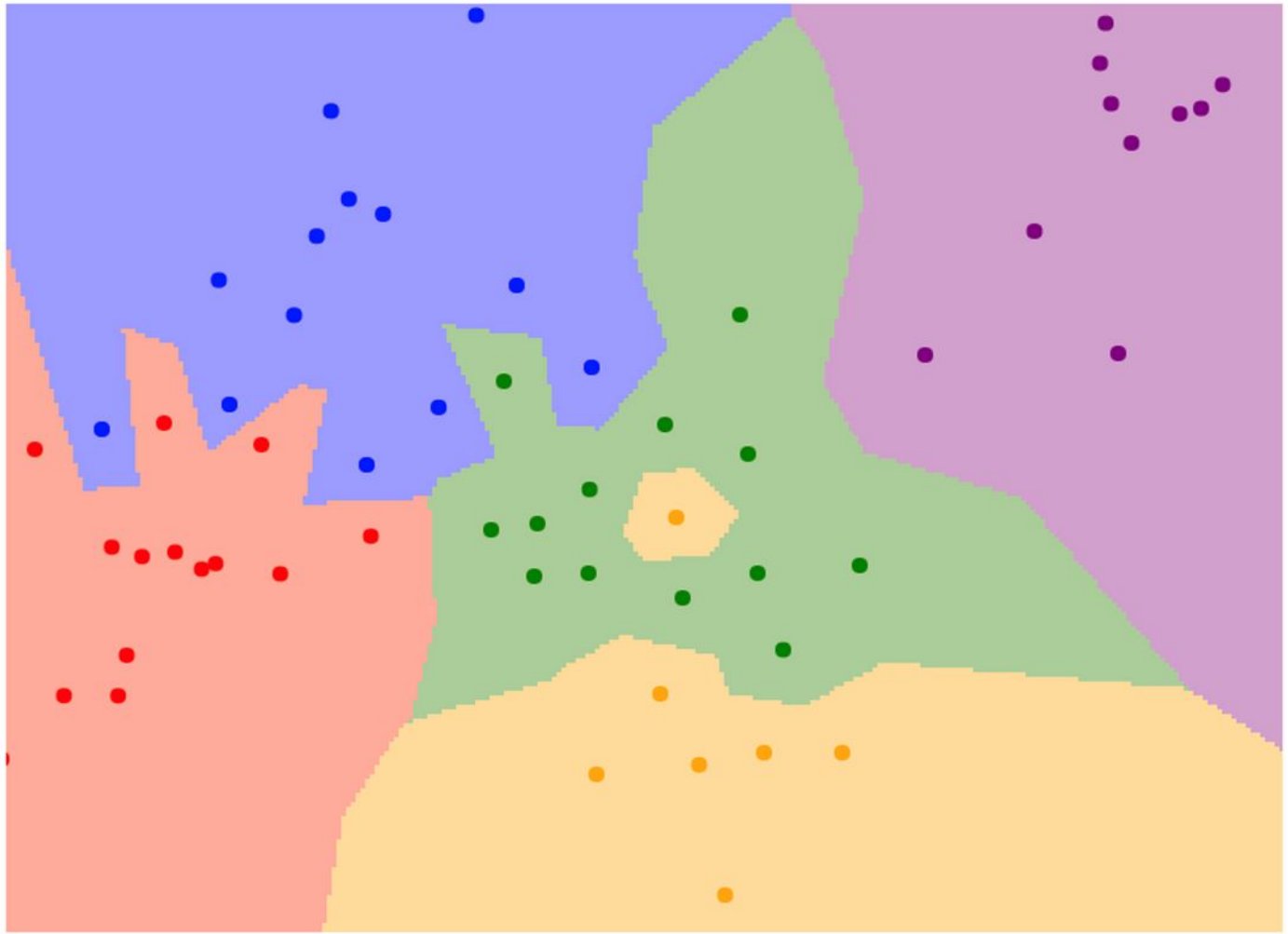
What does this look like?



What does this look like?

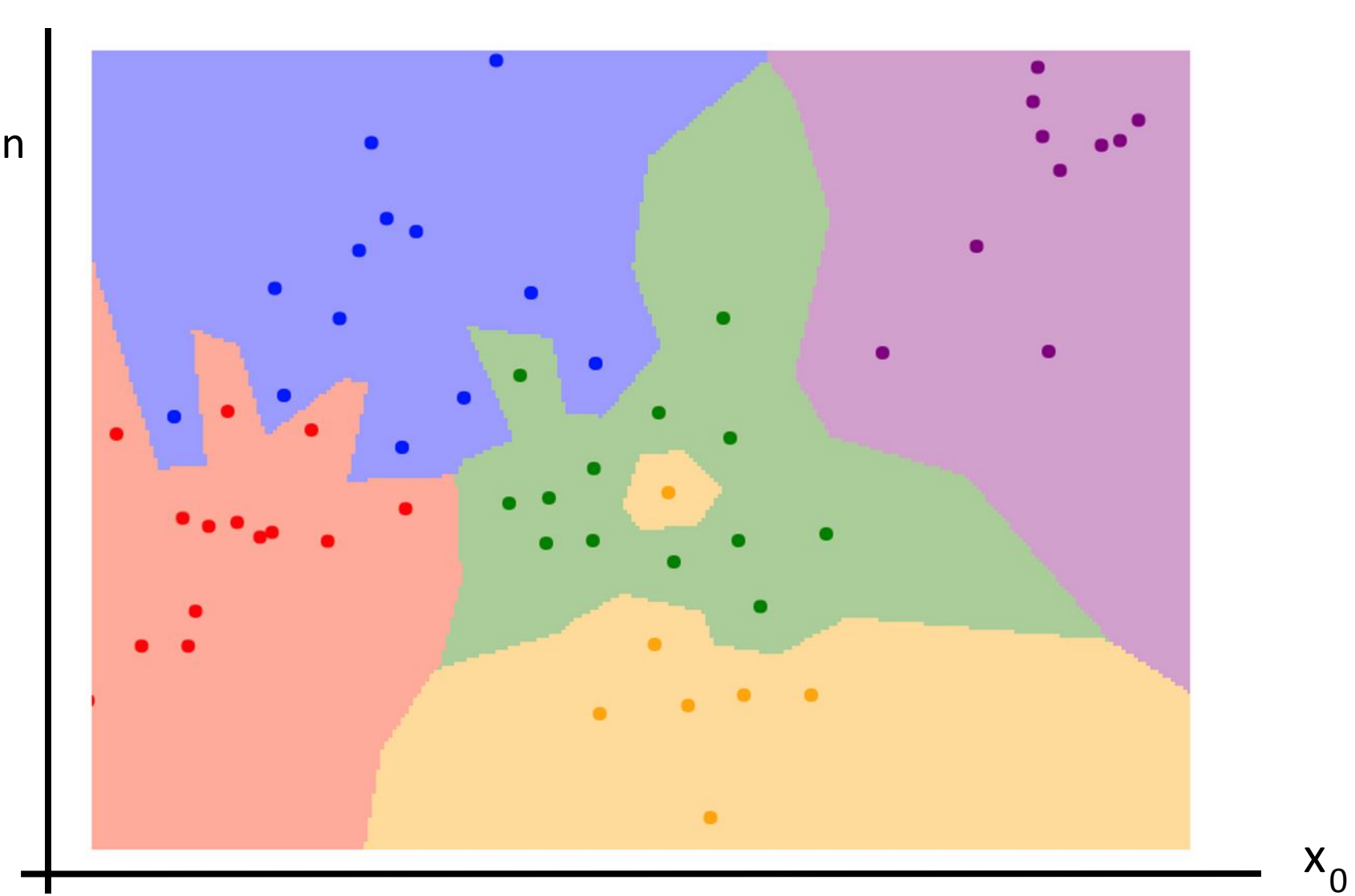


Nearest Neighbor Decision Boundaries



Nearest Neighbor Decision Boundaries

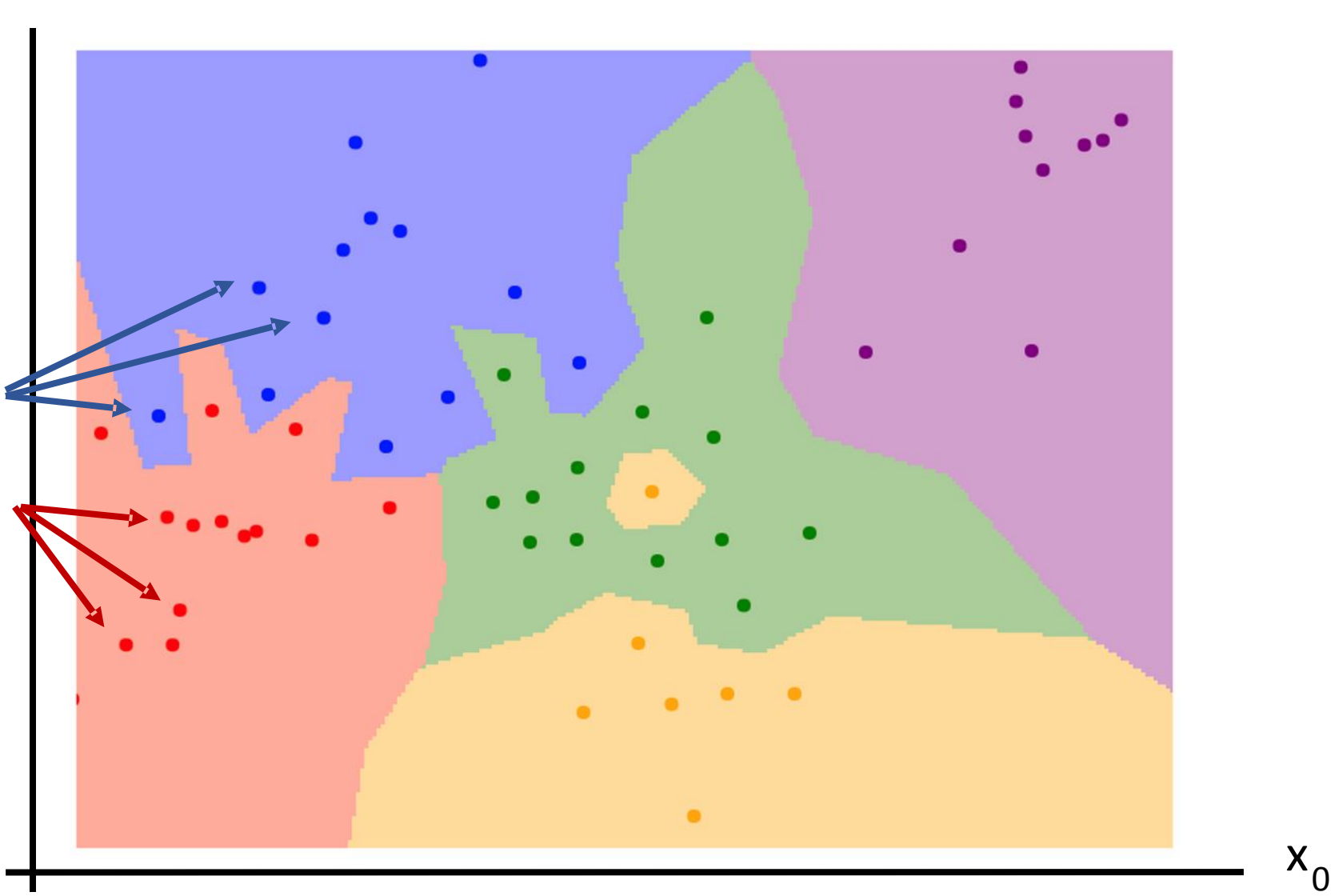
Nearest neighbors in two dimensions



Nearest Neighbor Decision Boundaries

Nearest neighbors in two dimensions

Points are training examples; colors give training labels

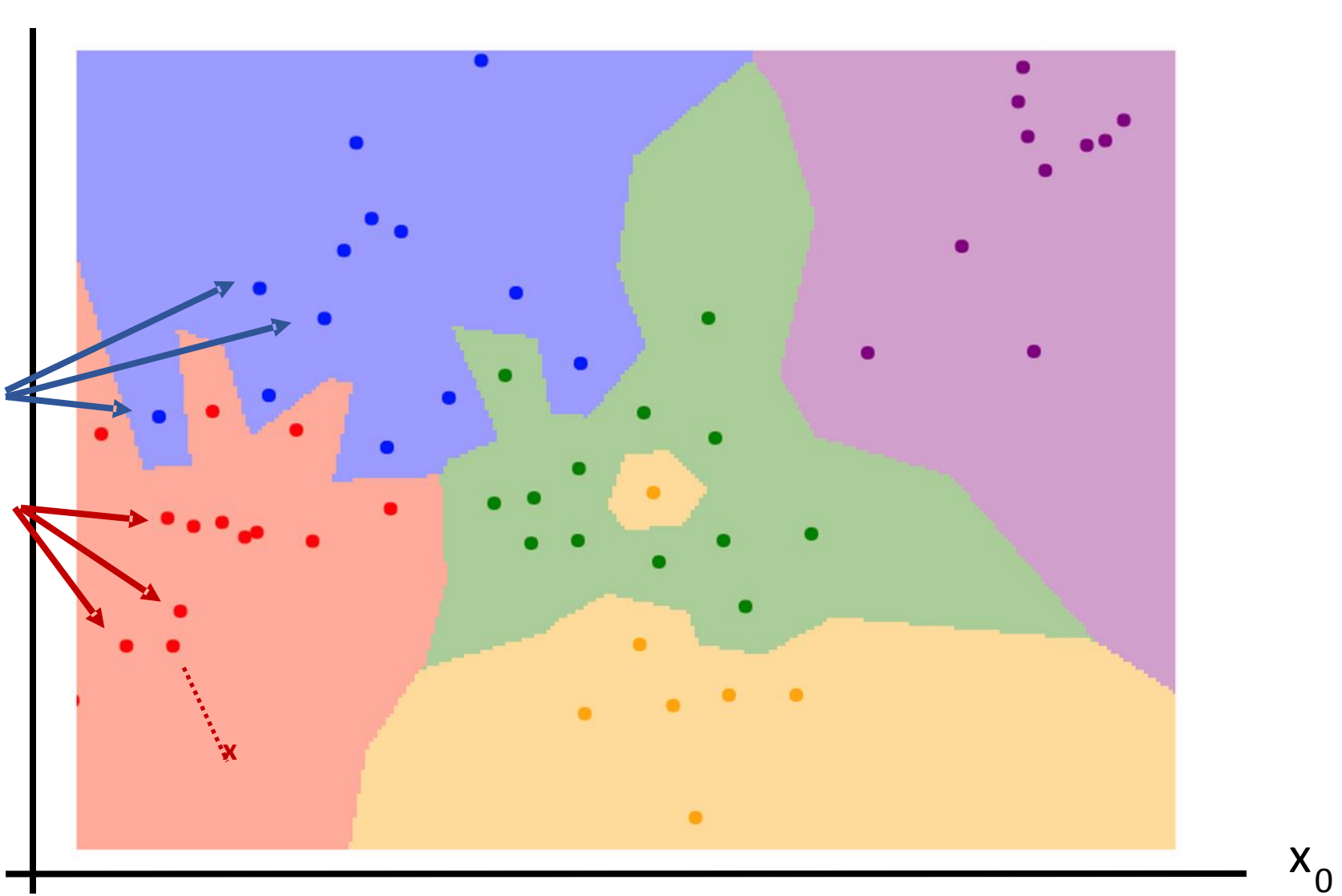


Nearest Neighbor Decision Boundaries

Nearest neighbors in two dimensions

Points are training examples; colors give training labels

Background colors give the category a test point would be assigned

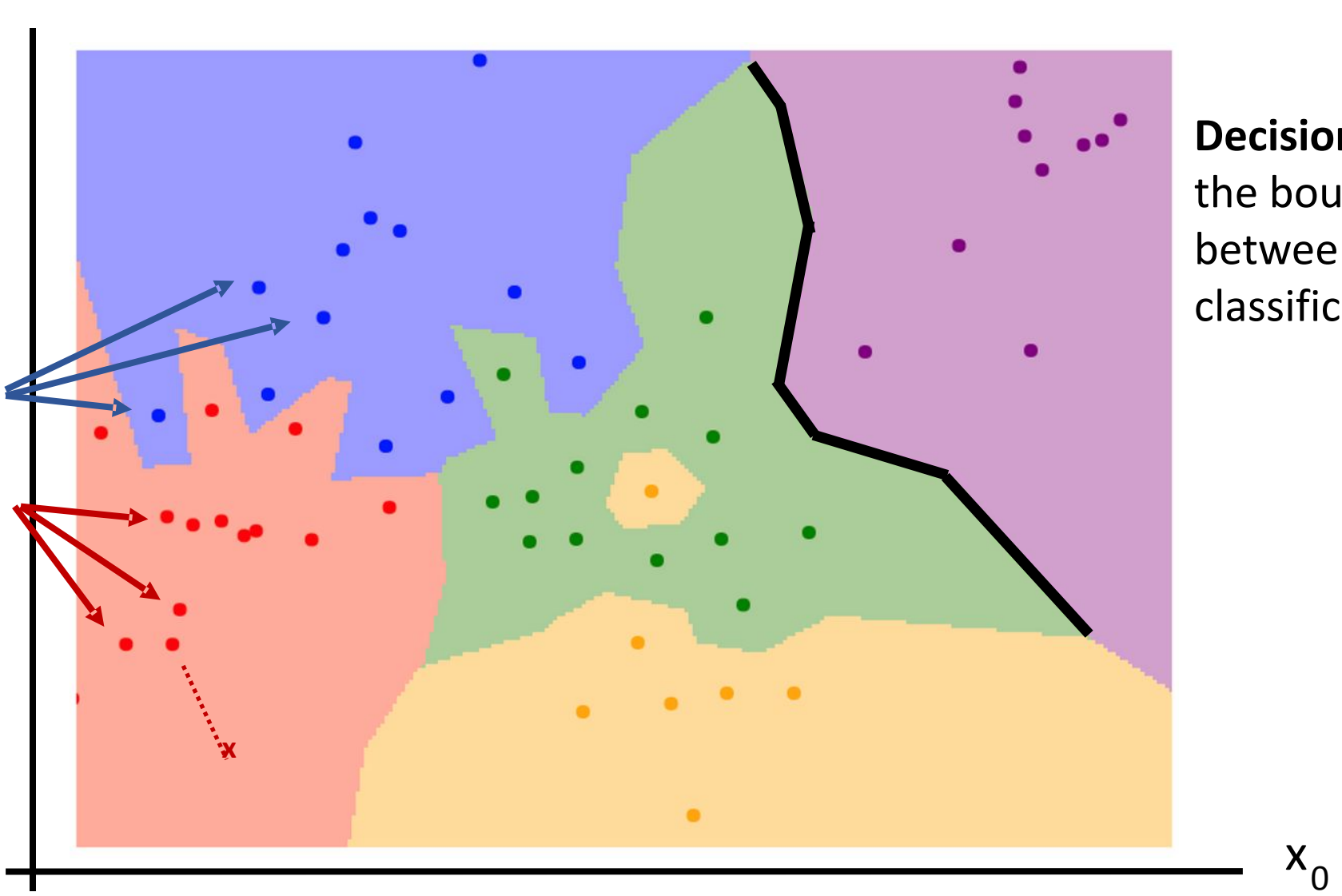


Nearest Neighbor Decision Boundaries

Nearest neighbors in two dimensions

Points are training examples; colors give training labels

Background colors give the category a test point would be assigned



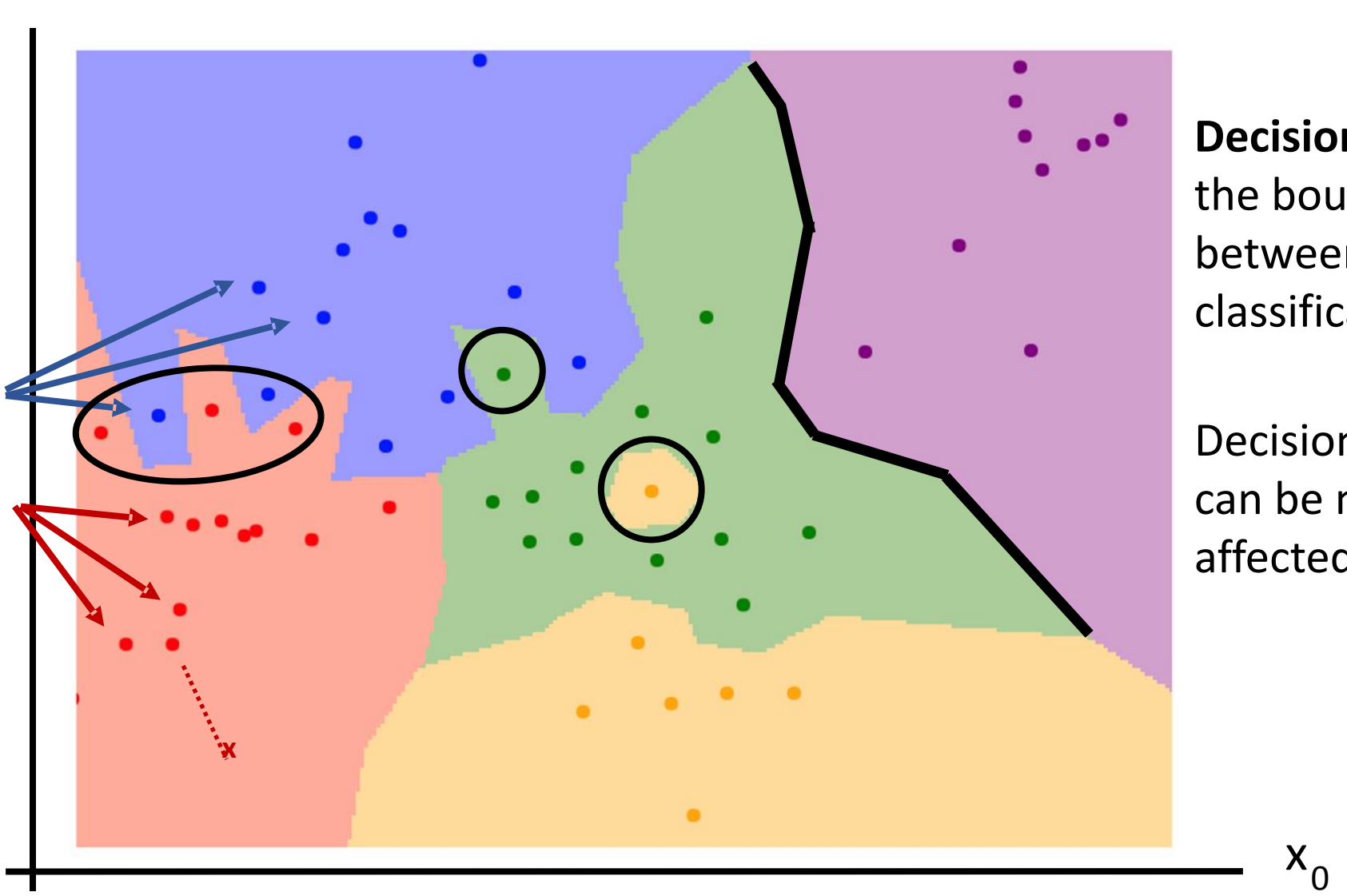
Decision boundary is the boundary between two classification regions

Nearest Neighbor Decision Boundaries

Nearest neighbors in two dimensions

Points are training examples; colors give training labels

Background colors give the category a test point would be assigned



Decision boundary is the boundary between two classification regions

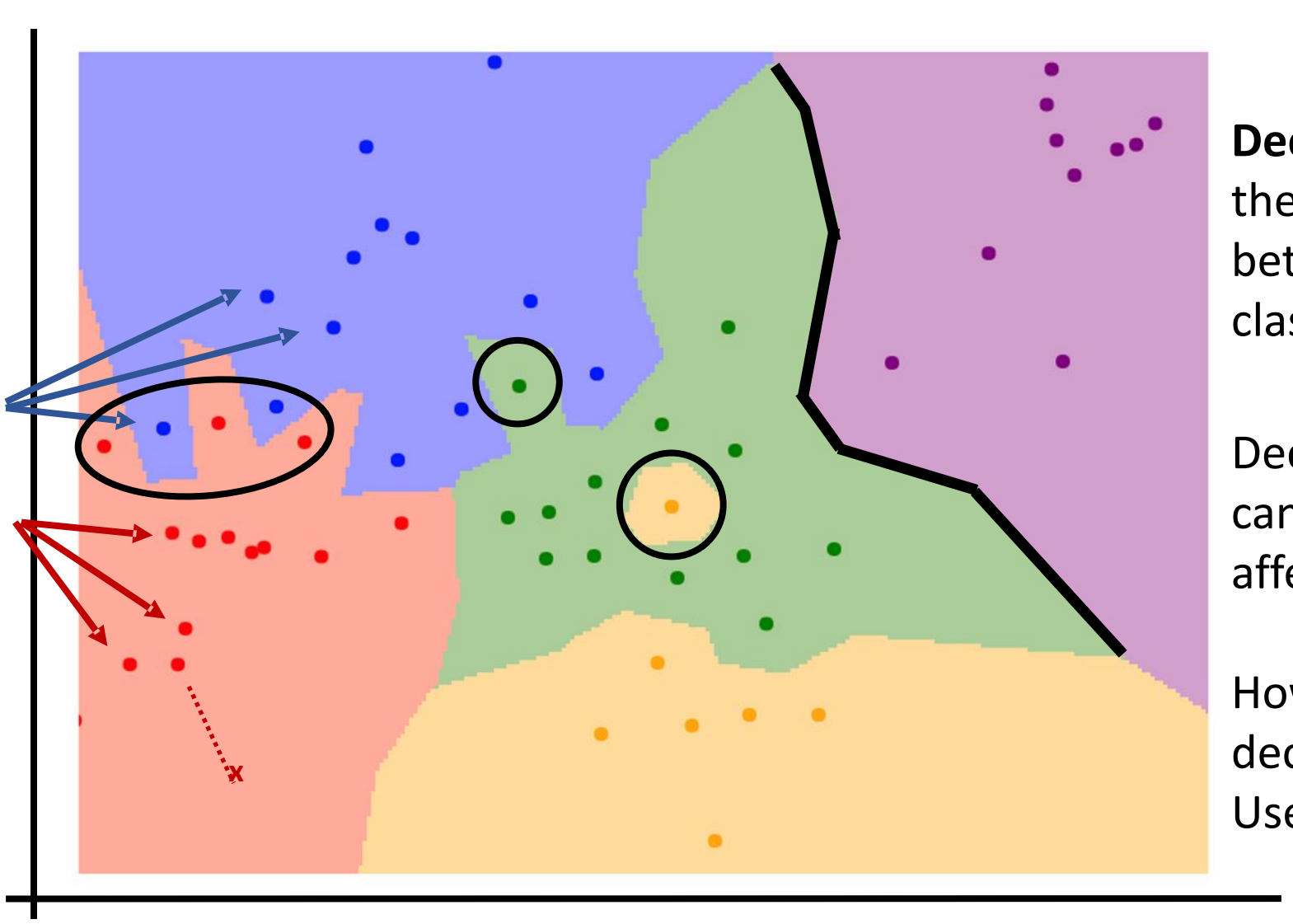
Decision boundaries can be noisy; affected by outliers

Nearest Neighbor Decision Boundaries

Nearest neighbors in two dimensions

Points are training examples; colors give training labels

Background colors give the category a test point would be assigned



Decision boundary is the boundary between two classification regions

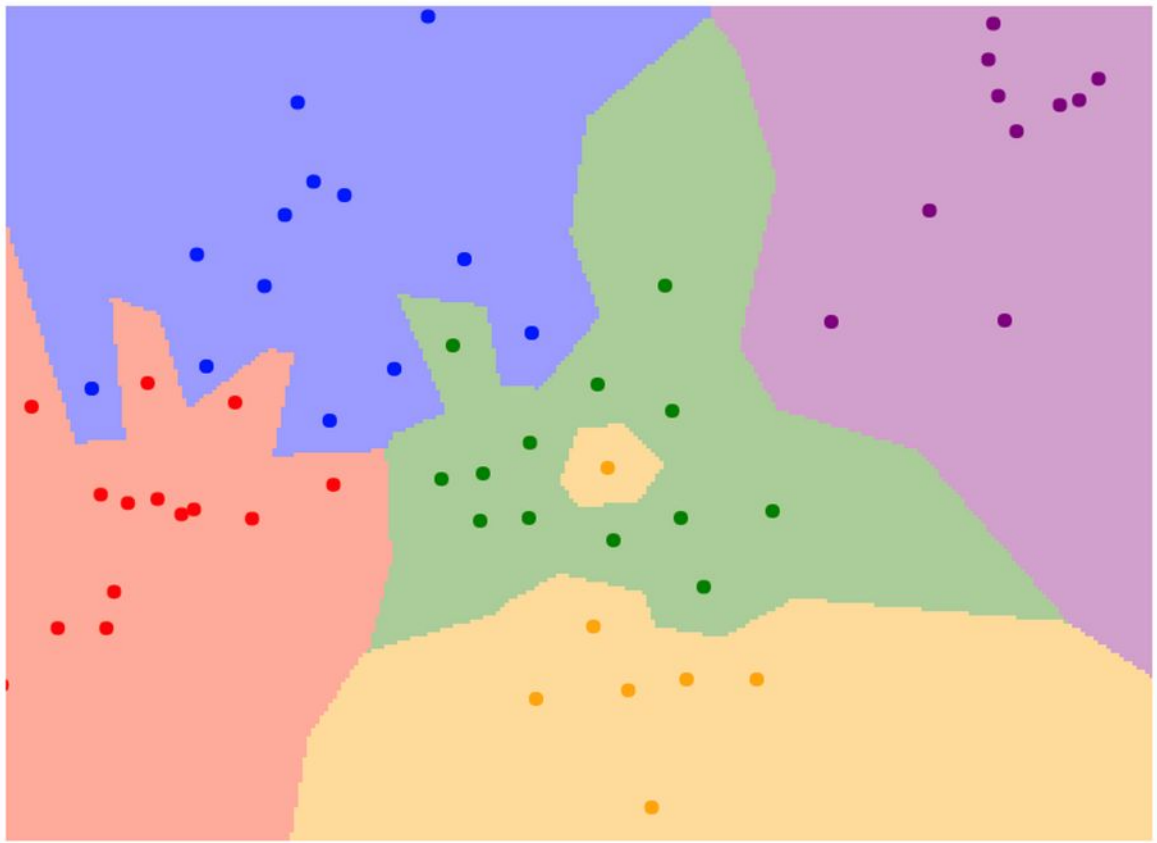
Decision boundaries can be noisy; affected by outliers

How to smooth out decision boundaries? Use more neighbors!

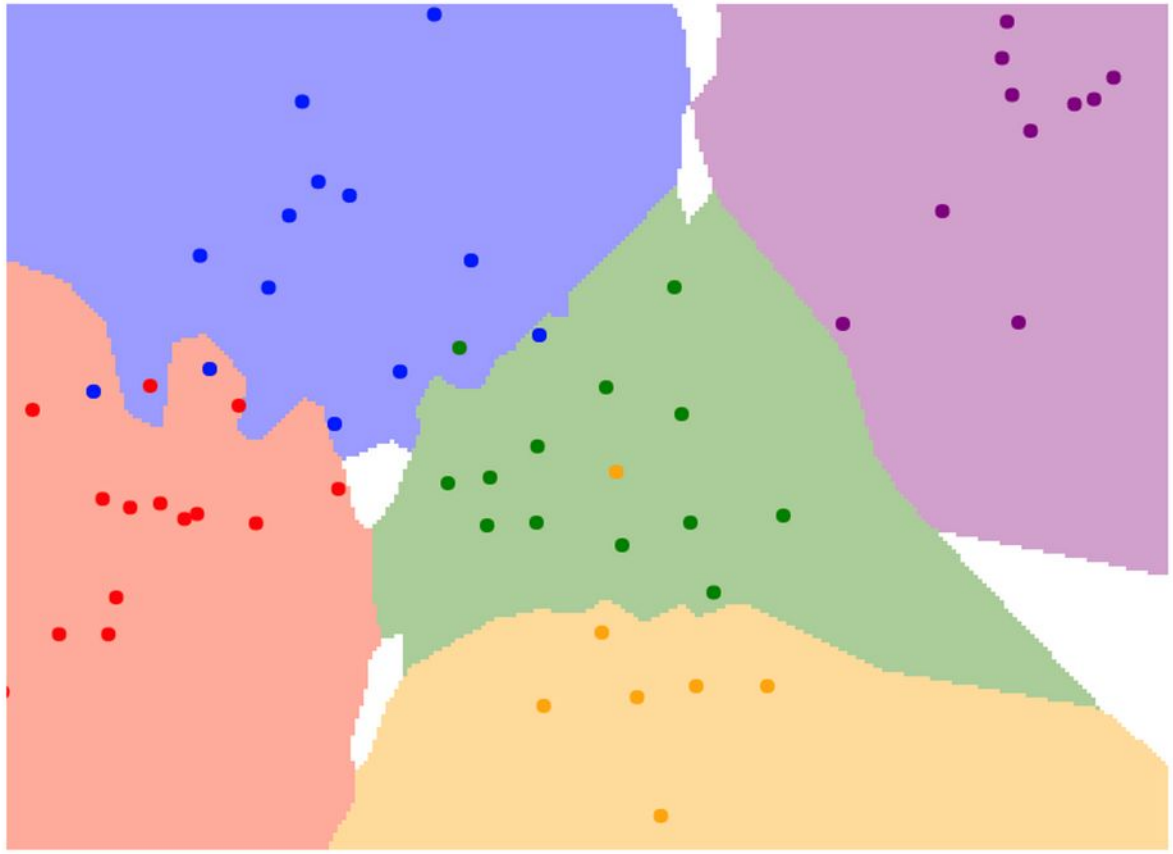
K-Nearest Neighbors

Instead of copying label from nearest neighbor, take **majority vote** from K closest points

K = 1



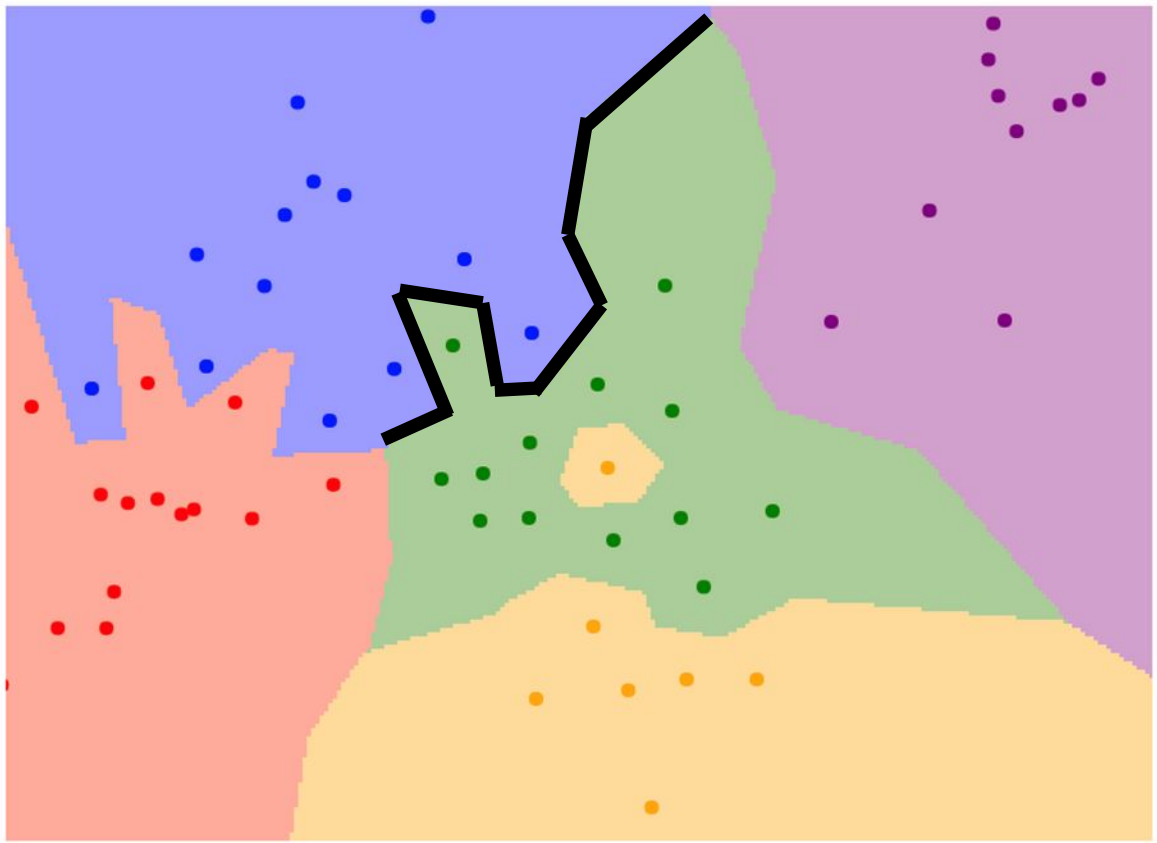
K = 3



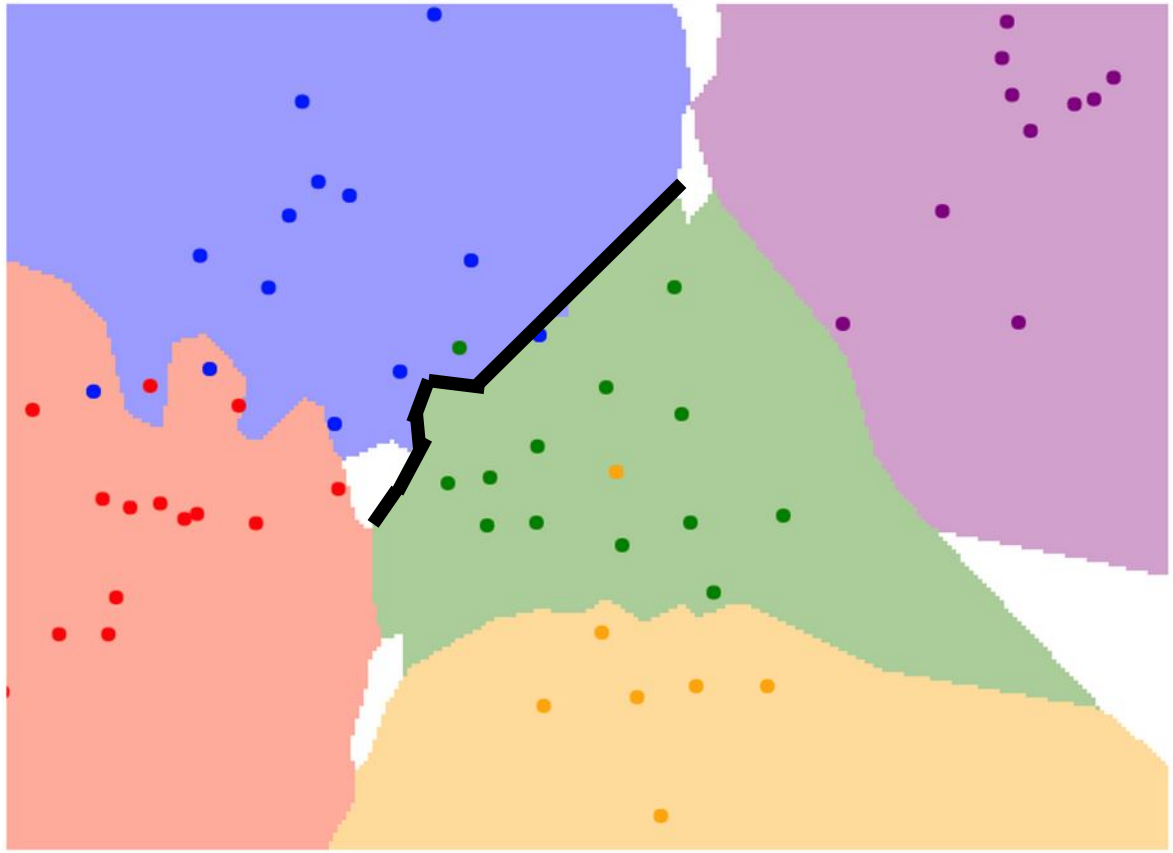
K-Nearest Neighbors

Using more neighbors helps smooth out rough decision boundaries

K = 1



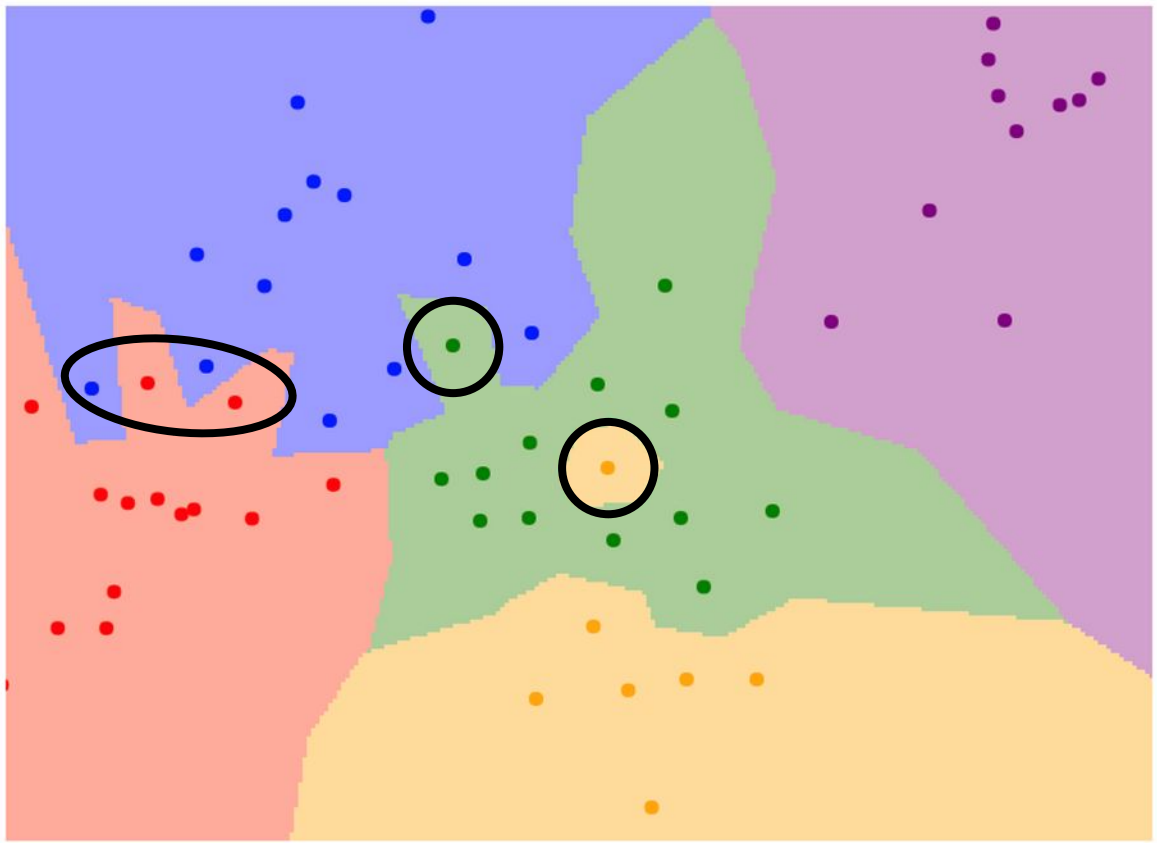
K = 3



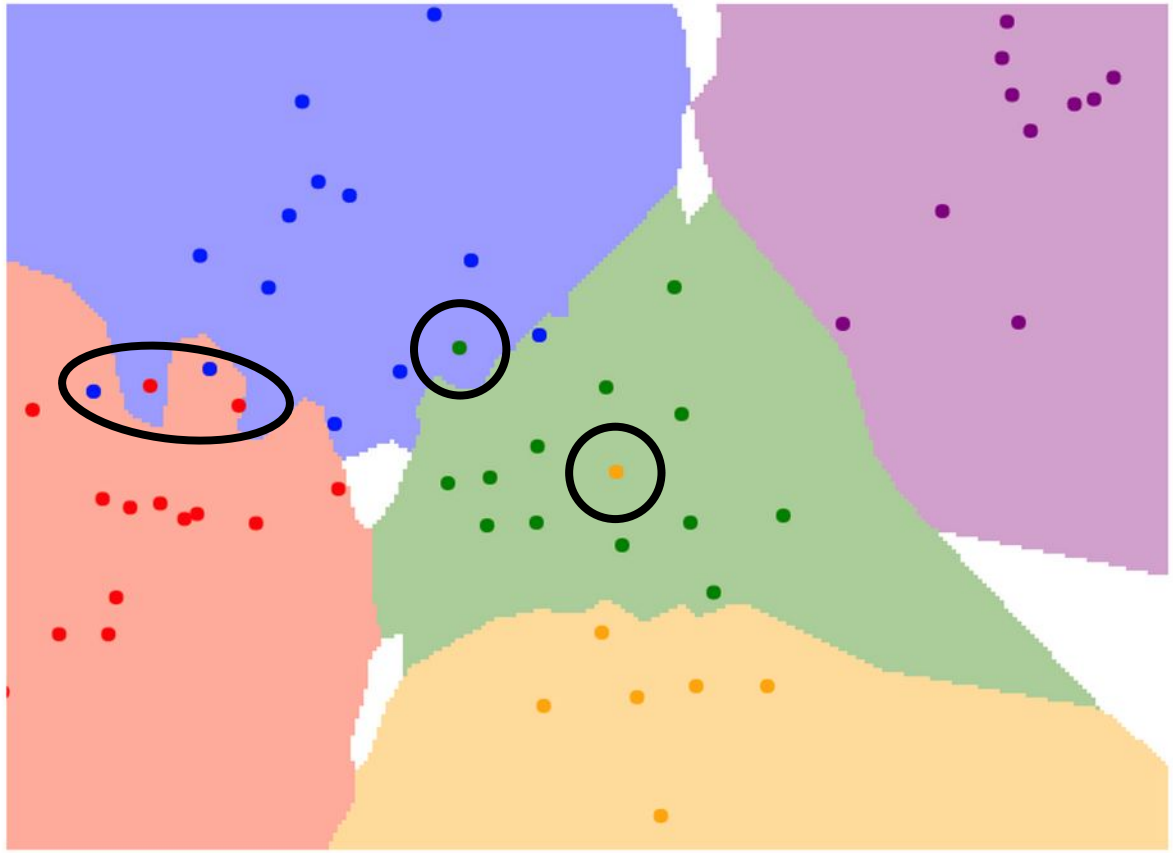
K-Nearest Neighbors

Using more neighbors helps reduce the effect of outliers

K = 1



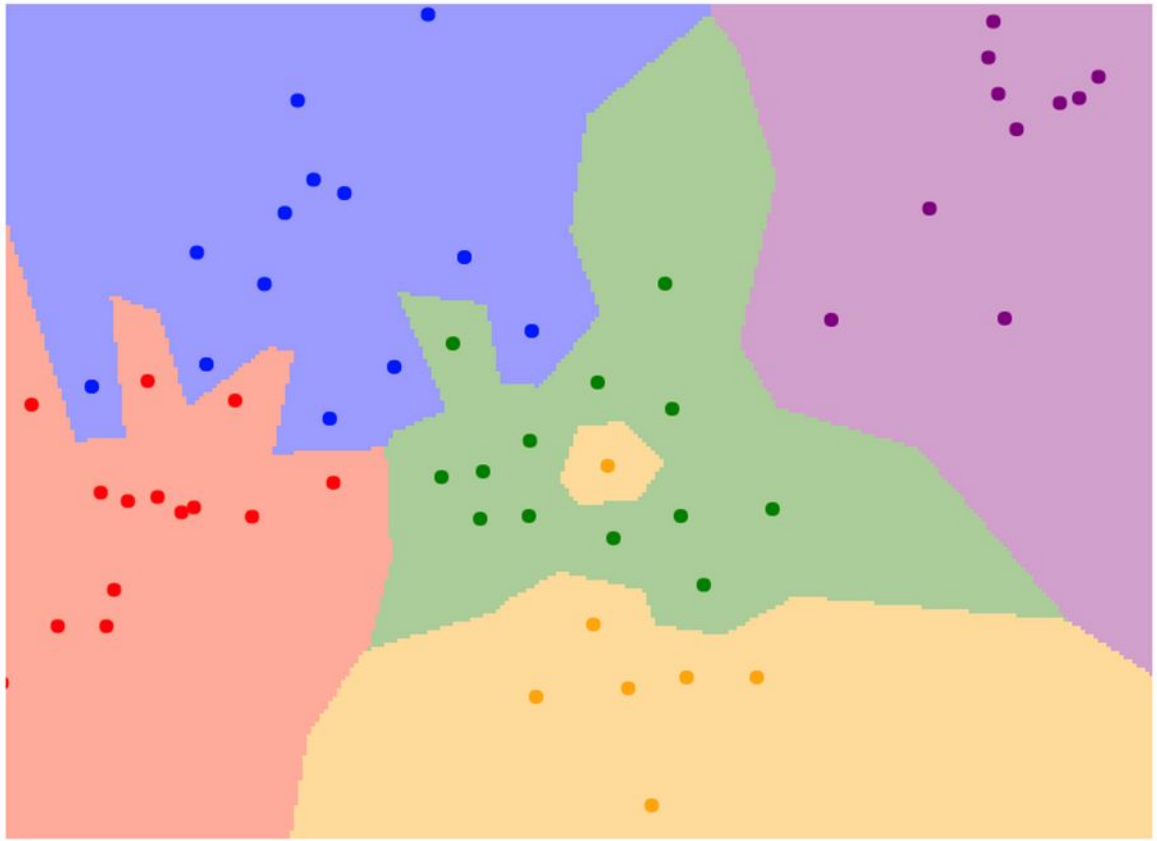
K = 3



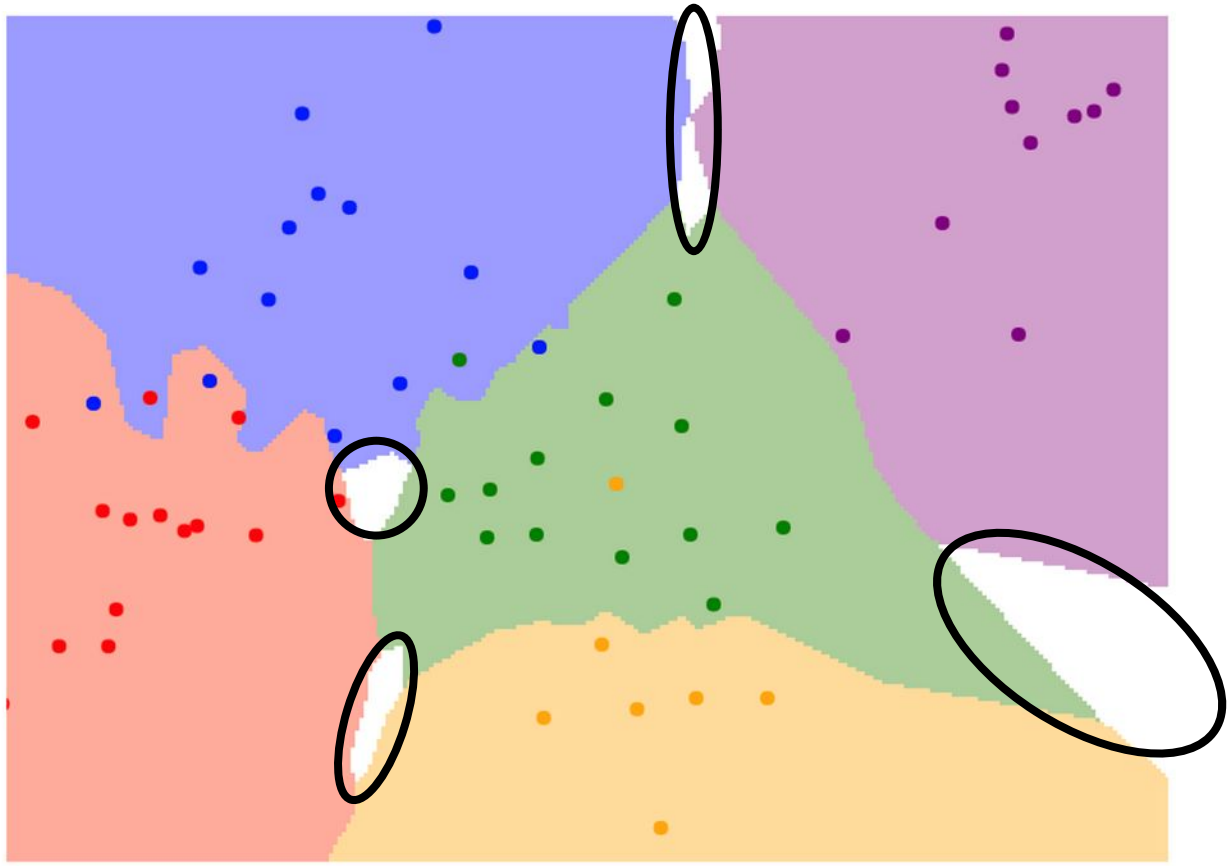
K-Nearest Neighbors

When K > 1 there can be ties between classes. Need to break somehow!

K = 1



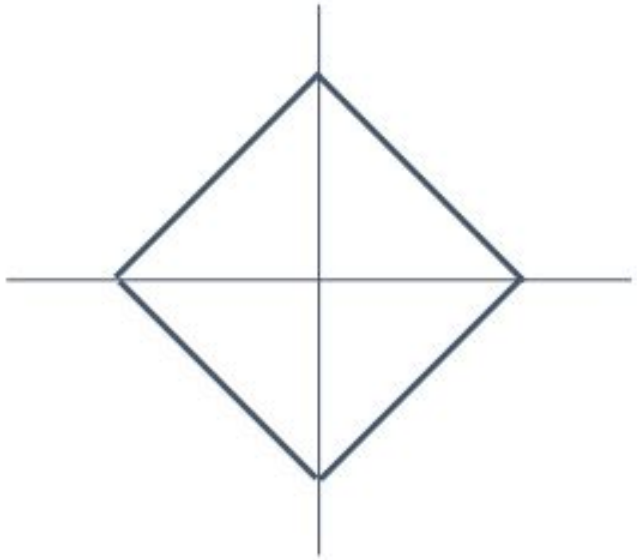
K = 3



K-Nearest Neighbors: Distance Metric

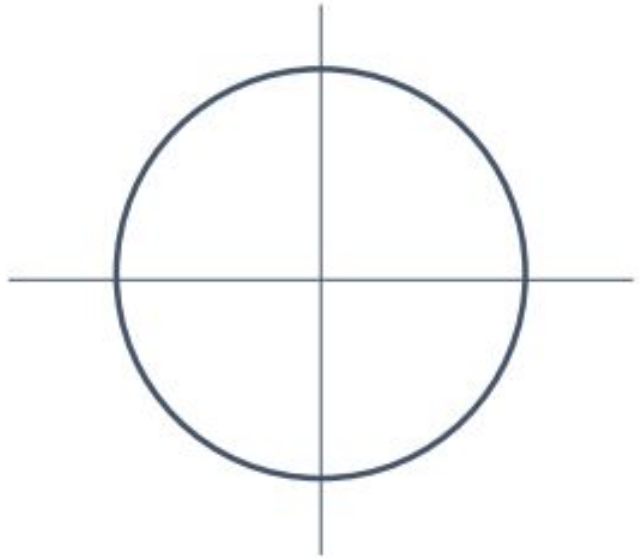
L1 (Manhattan) distance

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$



L2 (Euclidean) distance

$$d_1(I_1, I_2) = \left(\sum_p (I_1^p - I_2^p)^2 \right)^{\frac{1}{2}}$$



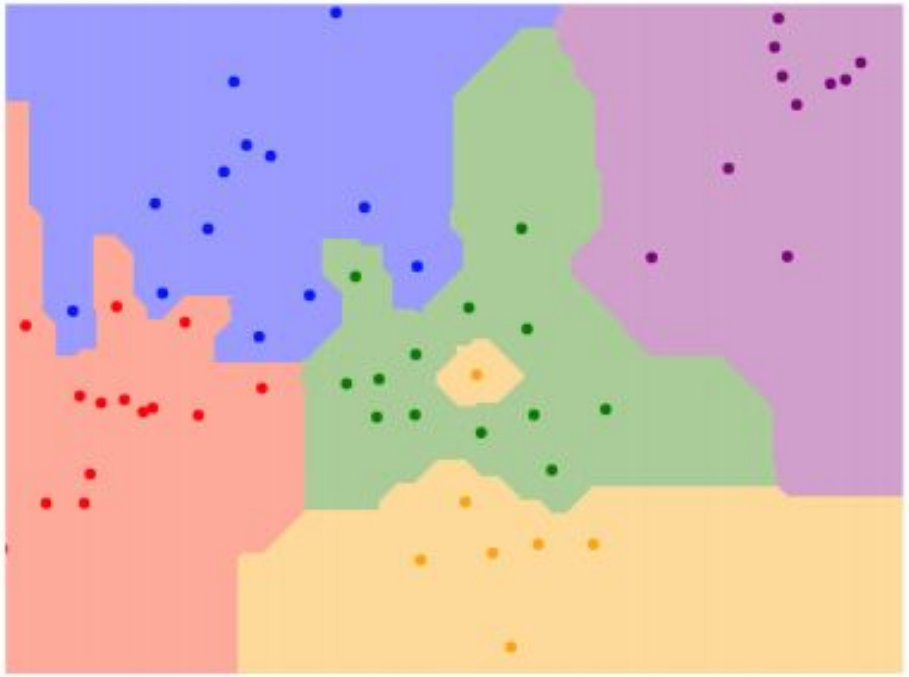
K-Nearest Neighbors: Distance Metric

L1 (Manhattan) distance

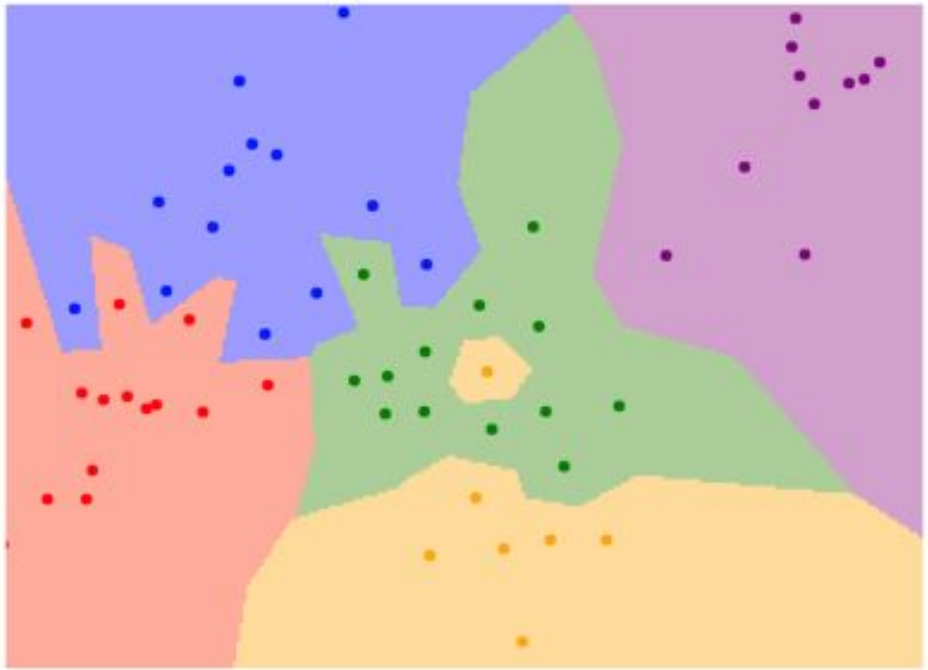
$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$

L2 (Euclidean) distance

$$d_1(I_1, I_2) = \left(\sum_p (I_1^p - I_2^p)^2 \right)^{\frac{1}{2}}$$



K = 1



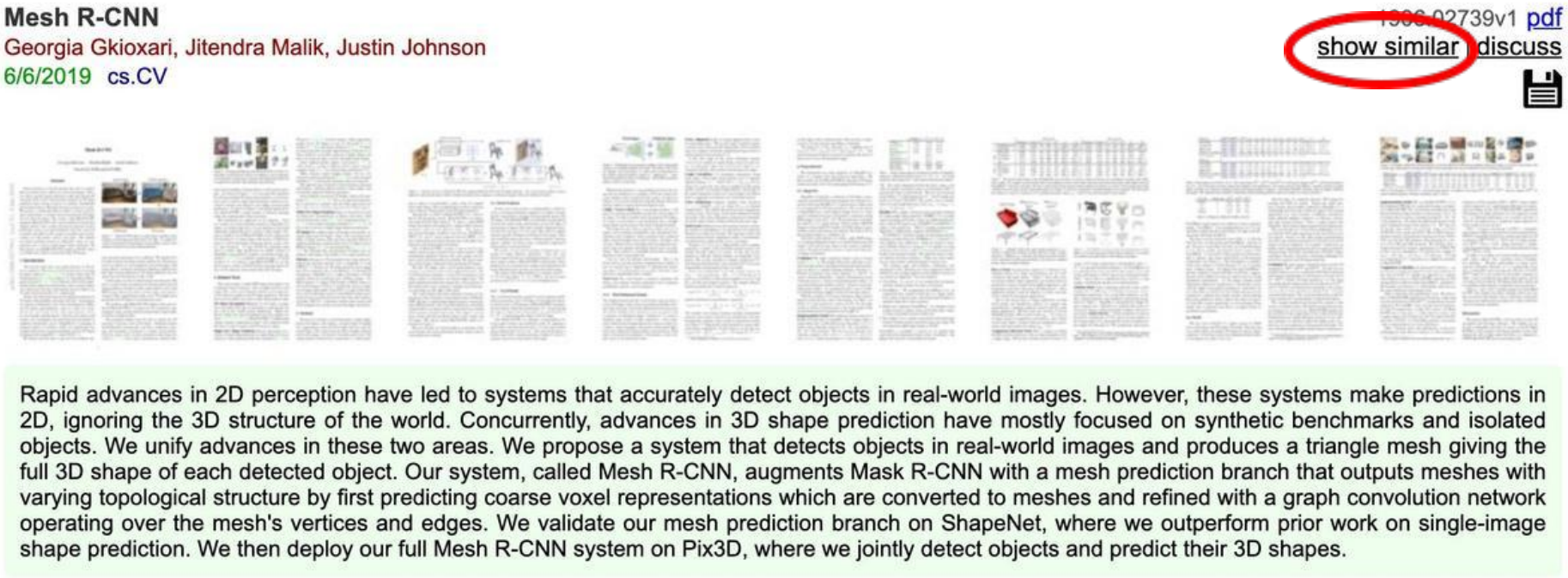
K-Nearest Neighbors: Distance Metric

With the right choice of distance metric, we can apply K-Nearest Neighbor to any type of data!

K-Nearest Neighbors: Distance Metric

With the right choice of distance metric, we can apply K-Nearest Neighbor to any type of data!

Example: Compare research papers using tf-idf similarity



Mesh R-CNN

Georgia Gkioxari, Jitendra Malik, Justin Johnson

6/6/2019 cs.CV

1906.02739v1 pdf

show similar | discuss

Rapid advances in 2D perception have led to systems that accurately detect objects in real-world images. However, these systems make predictions in 2D, ignoring the 3D structure of the world. Concurrently, advances in 3D shape prediction have mostly focused on synthetic benchmarks and isolated objects. We unify advances in these two areas. We propose a system that detects objects in real-world images and produces a triangle mesh giving the full 3D shape of each detected object. Our system, called Mesh R-CNN, augments Mask R-CNN with a mesh prediction branch that outputs meshes with varying topological structure by first predicting coarse voxel representations which are converted to meshes and refined with a graph convolution network operating over the mesh's vertices and edges. We validate our mesh prediction branch on ShapeNet, where we outperform prior work on single-image shape prediction. We then deploy our full Mesh R-CNN system on Pix3D, where we jointly detect objects and predict their 3D shapes.

Hyperparameters

What is the best value of **K** to use?
What is the best **distance metric** to use?

These are examples of **hyperparameters**: choices about our learning algorithm that we don't learn from the training data; instead we set them at the start of the learning process

Hyperparameters

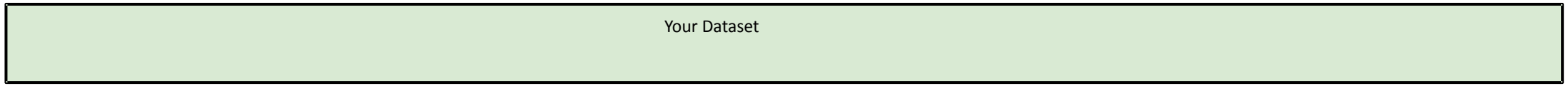
What is the best value of **K** to use?
What is the best **distance metric** to use?

These are examples of **hyperparameters**: choices about our learning algorithm that we don't learn from the training data; instead we set them at the start of the learning process

Very problem-dependent. In general need to try them all and see what works best for our data / task.

Setting Hyperparameters

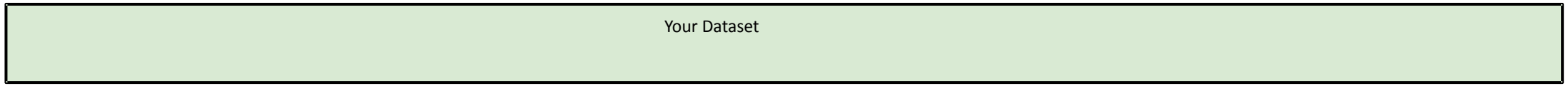
Idea #1: Choose hyperparameters that work best on the data



Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

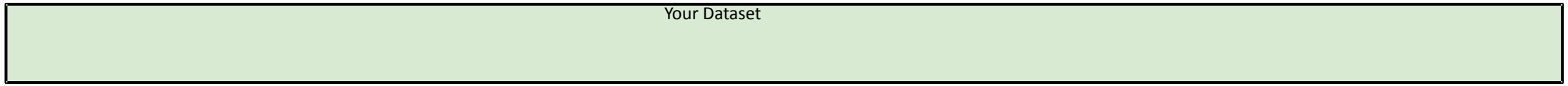
BAD: K = 1 always works perfectly on training data



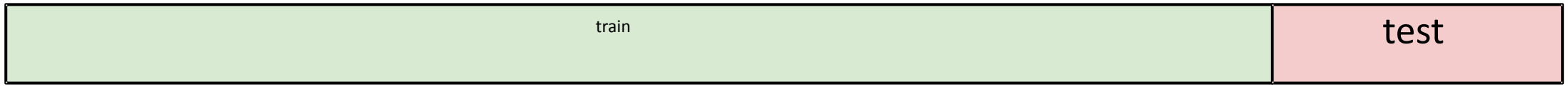
Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

BAD: K = 1 always works perfectly on training data



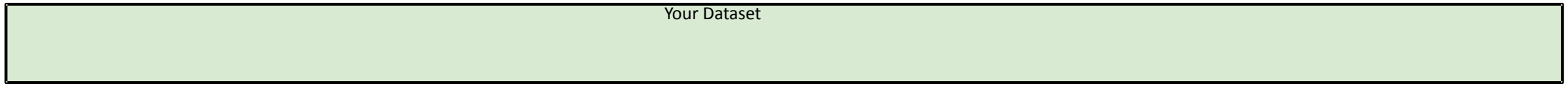
Idea #2: Split data into **train** and **test**, choose hyperparameters that work best on test data



Setting Hyperparameters

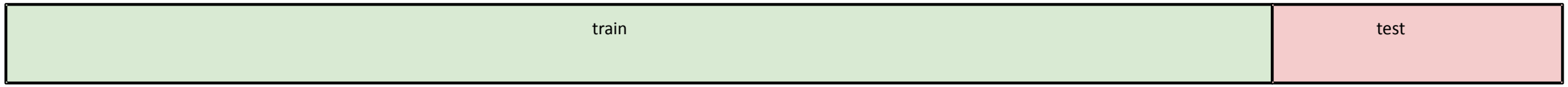
Idea #1: Choose hyperparameters that work best on the data

BAD: K = 1 always works perfectly on training data



Idea #2: Split data into **train** and **test**, choose hyperparameters that work best on test data

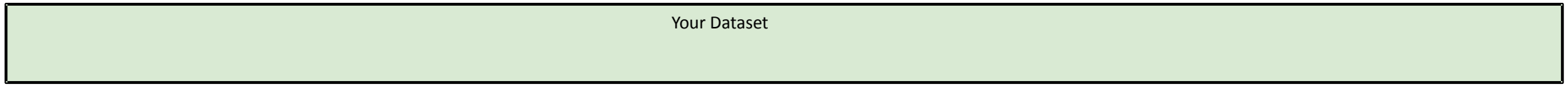
BAD: No idea how algorithm will perform on new data



Setting Hyperparameters

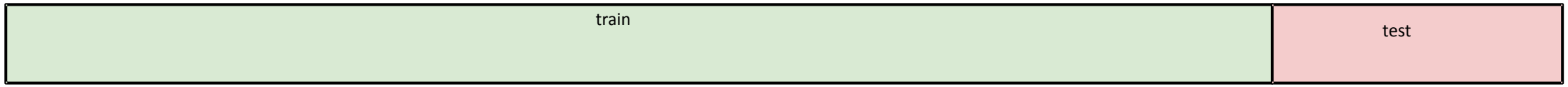
Idea #1: Choose hyperparameters that work best on the data

BAD: K = 1 always works perfectly on training data



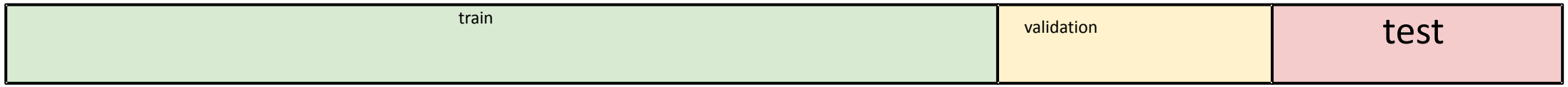
Idea #2: Split data into **train** and **test**, choose hyperparameters that work best on test data

BAD: No idea how algorithm will perform on new data

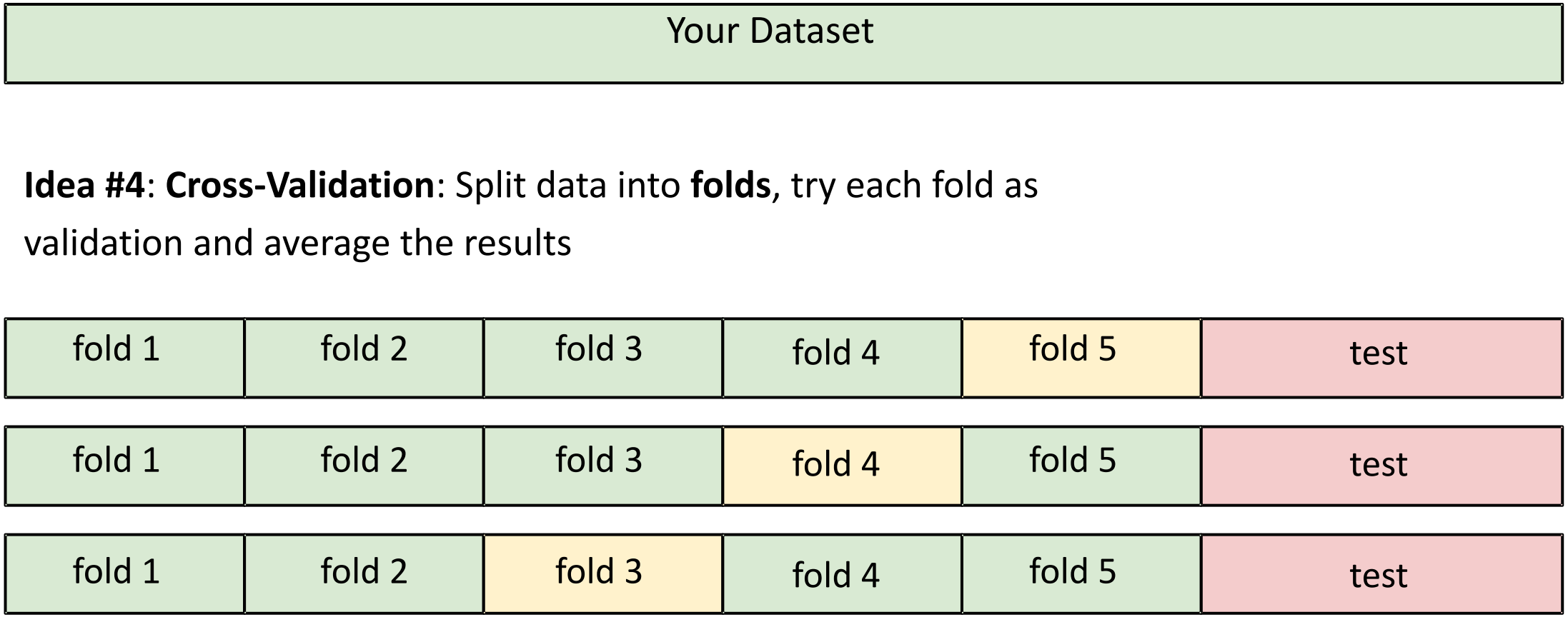


Idea #3: Split data into **train**, **val**, and **test**; choose hyperparameters on val and evaluate on test

Better!



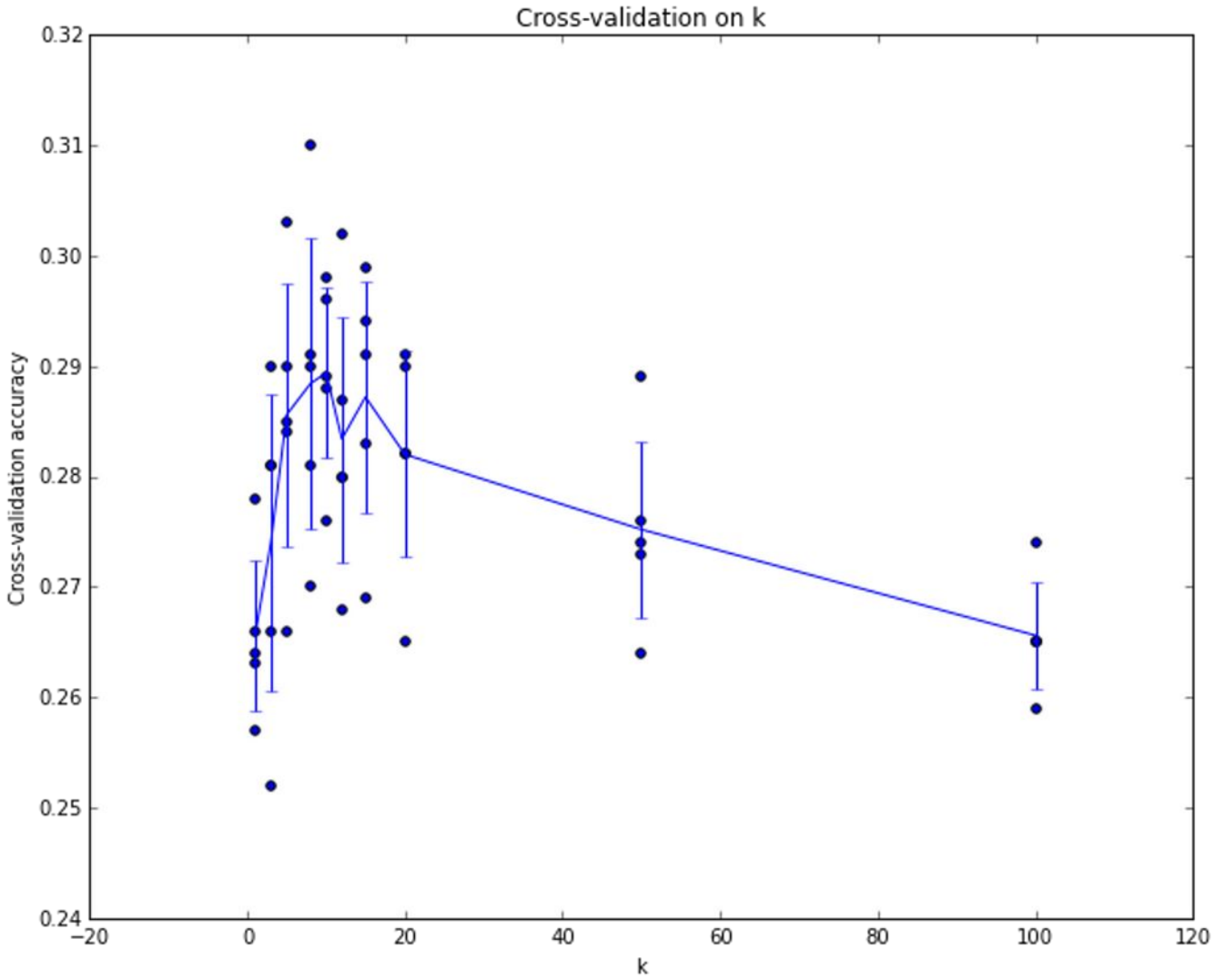
Setting Hyperparameters



Idea #4: **Cross-Validation**: Split data into **folds**, try each fold as validation and average the results

Useful for small datasets, but (unfortunately) not used too frequently in deep learning

Setting Hyperparameters



Example of 5-fold cross-validation for the value of **k.**

Each point: single outcome.

The line goes through the mean, bars indicated standard deviation

(Seems that k ~ 7 works best for this data)

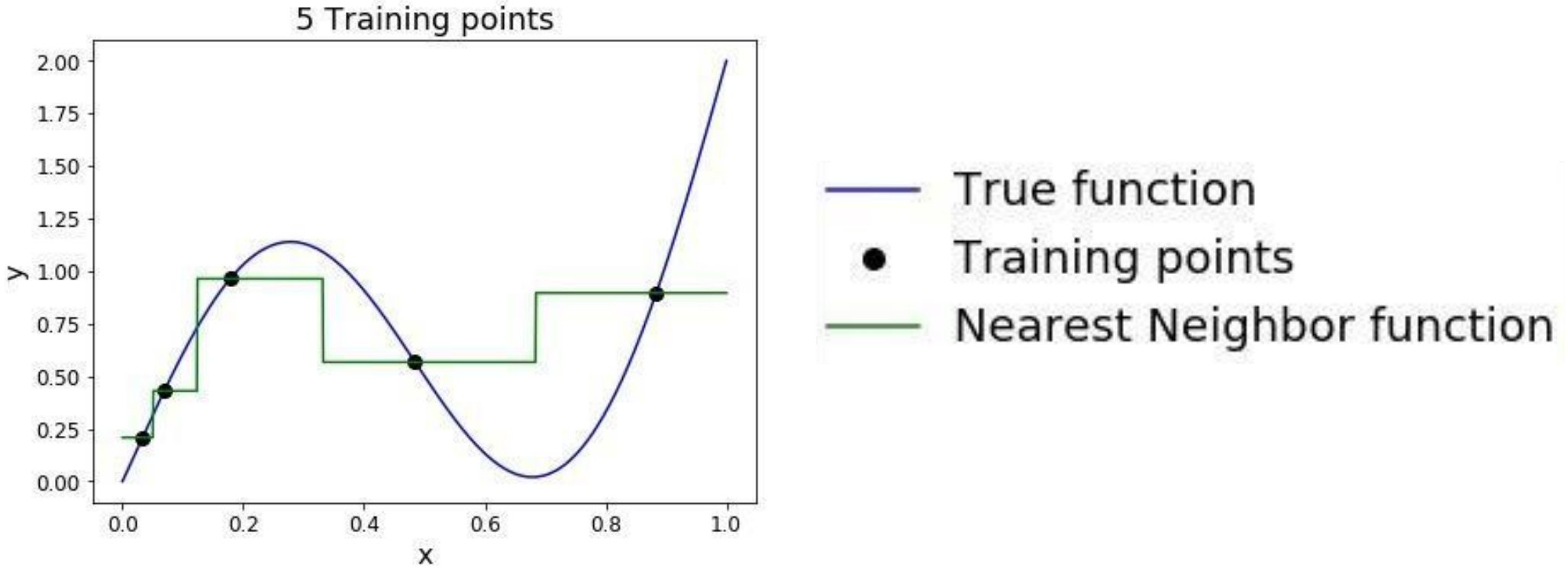
K-Nearest Neighbor: Universal Approximation

As the number of training samples goes to infinity, nearest neighbor can represent any (*) function!

(*) Subject to many technical conditions. Only continuous functions on a compact domain.

K-Nearest Neighbor: Universal Approximation

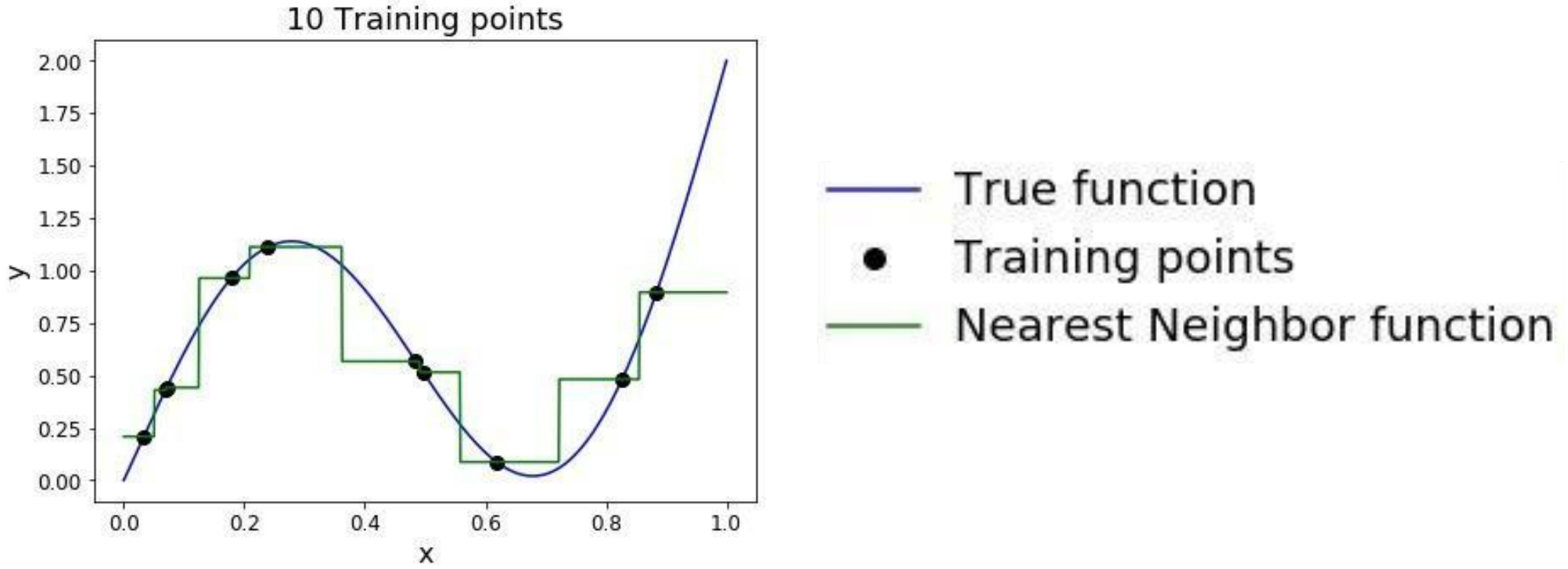
As the number of training samples goes to infinity, nearest neighbor can represent any (*) function!



(*) Subject to many technical conditions. Only continuous functions on a compact domain.

K-Nearest Neighbor: Universal Approximation

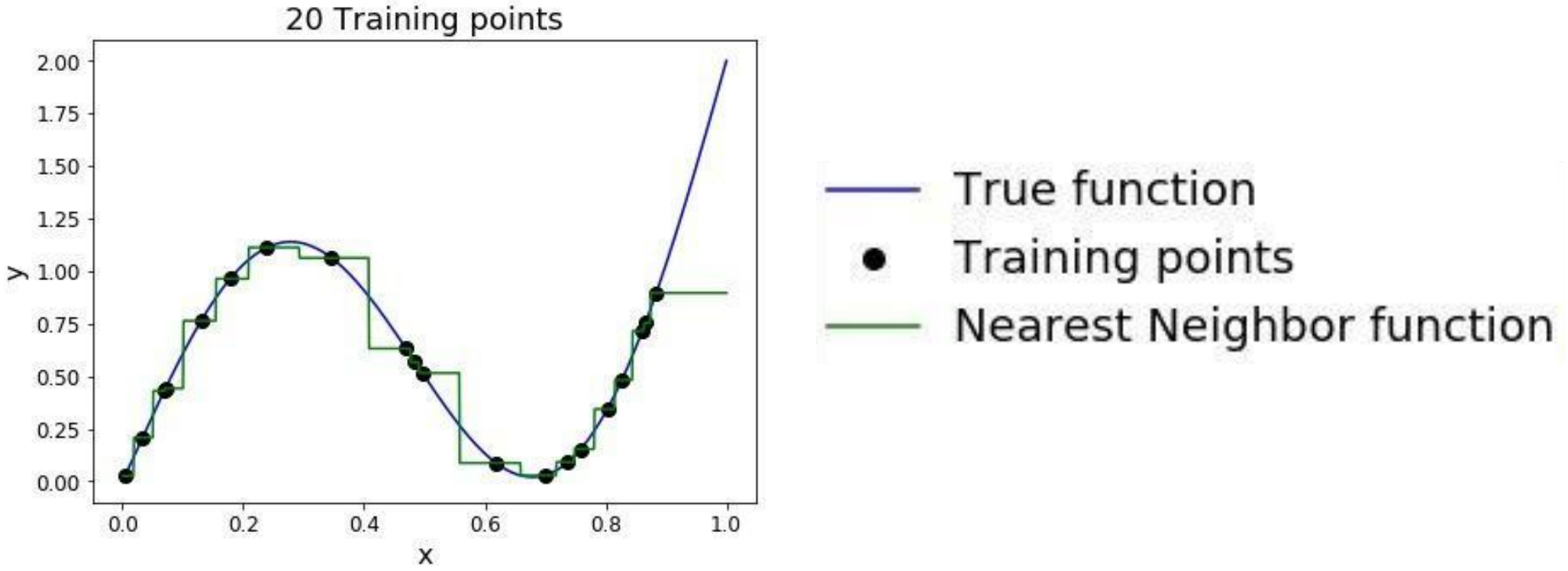
As the number of training samples goes to infinity, nearest neighbor can represent any (*) function!



(*) Subject to many technical conditions. Only continuous functions on a compact domain.

K-Nearest Neighbor: Universal Approximation

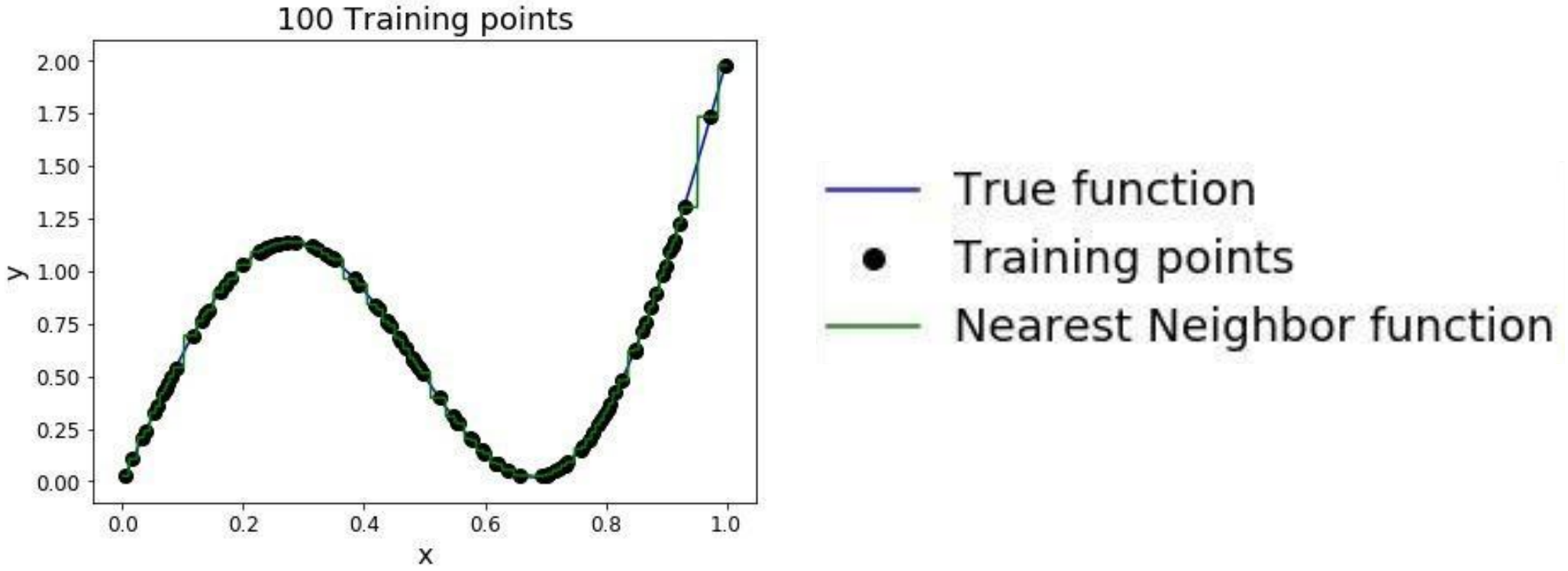
As the number of training samples goes to infinity, nearest neighbor can represent any (*) function!



(*) Subject to many technical conditions. Only continuous functions on a compact domain.

K-Nearest Neighbor: Universal Approximation

As the number of training samples goes to infinity, nearest neighbor can represent any (*) function!



(*) Subject to many technical conditions. Only continuous functions on a compact domain.

Problem: Curse of Dimensionality

Uniform coverage requires exponentially more training points as dimensionality increases.

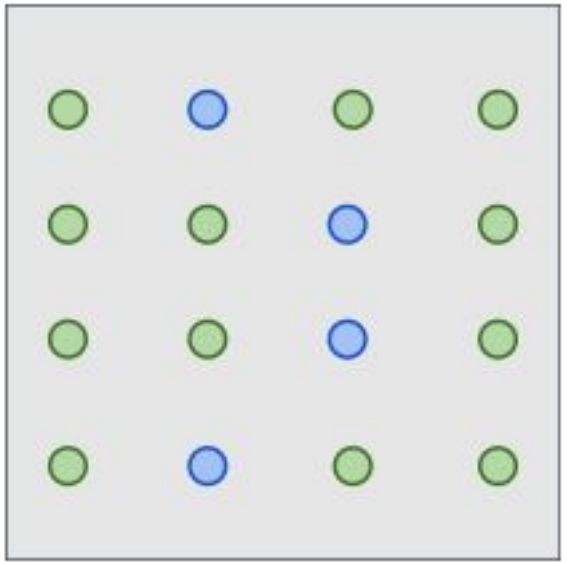
Dimensions = 1

Points = 4



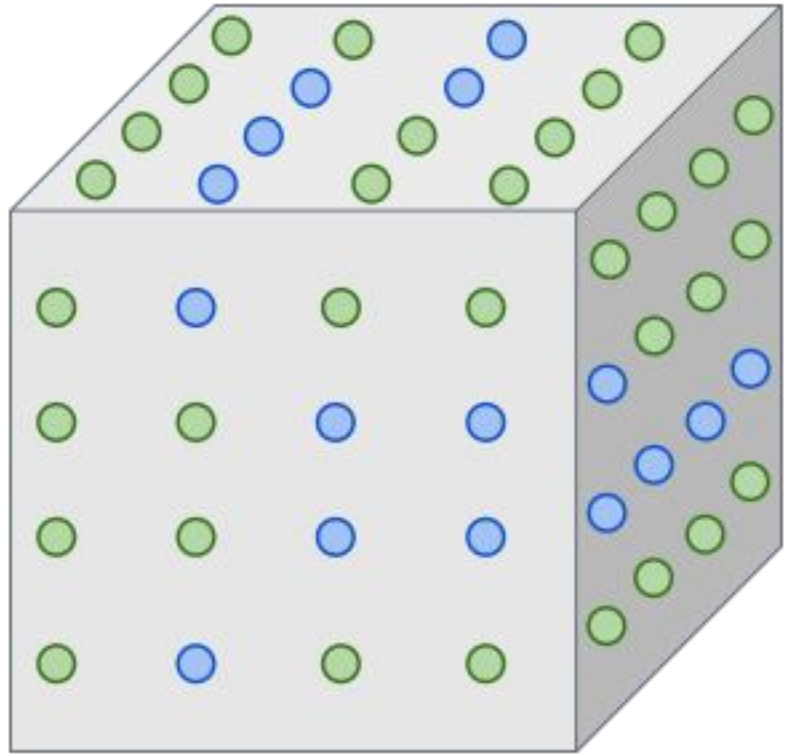
Dimensions = 2

Points = 42



Dimensions = 3

Points = 43



Problem: Curse of Dimensionality

Curse of dimensionality: For uniform coverage of space, number of training points needed grows exponentially with dimension

Number of possible 32x32 binary images:

$$2^{32x32} \approx 10^{308}$$

Problem: Curse of Dimensionality

Uniform coverage requires exponentially more training points as dimensionality increases.

Number of possible 32x32 binary images:

$2^{32x32} \approx 10^{308}$

Number of elementary particles in the visible universe: (source)

$\approx 10^{97}$

K-Nearest Neighbor on raw pixels is seldom used

- Very slow at test time
- Distance metrics on pixels are not informative

Original | Boxed | Shifted | Tinted



(all 3 images have same L2 distance to the one on the left)

Original image is CC0 public domain

Nearest Neighbor with ConvNet features works well!



Devlin et al, “Exploring Nearest Neighbor Approaches for Image Captioning”

Nearest Neighbor with ConvNet features works well!

Example: Image Captioning with Nearest Neighbor



A bedroom with a bed and a couch.



A cat sitting in a bathroom sink.



A train is stopped at a train station.



A wooden bench in front of a building.

Devlin et al, “Exploring Nearest Neighbor Approaches for Image Captioning”,

Summary

In image classification we start with a **training set** of images and labels, and must predict labels on the **test set**
Image classification is challenging due to the semantic gap: we need invariance to occlusion, deformation, lighting, intraclass variation, etc

Image classification is a **building block** for other vision tasks

The **K-Nearest Neighbors** classifier predicts labels based on nearest training examples

Distance metric and K are **hyperparameters**

Choose hyperparameters using the **validation set**; only run on the test set once at the very end!

Next time: Linear Classifiers

